

Technical Assignment Report

Generation of a Variable Duty Cycle PWM Signal via DAC using
AT89S52

Microprocessor and Microcontroller Interfacing

Submitted by:

Name	Roll Number
Sahil Sharma	002311102030
Agasta Das	002311102027
Supriyo Banerjee	002311102028
Supritam Kar	002311102029

*A detailed analysis of hardware configurations, baud rate mapping, DAC characteristics,
and software timer architecture.*

April 12, 2026

Contents

1	Introduction and Objective	2
2	Hardware Characteristics & Clock Assumptions	2
3	Serial Communication & Baud Rate Calculations	2
4	PWM Timer Period & Frequency Constraints	3
5	DAC Derivations & Voltage Calculus	3
5.1	Proving +2V Output Limit	4
5.2	Proving -2V Output Limit	4
5.3	Current-to-Voltage Equivalent	4
6	PWM Generation Assembly Logic	4
A	Hardware Output Waveform	5
B	Microcontroller Source Code	6
C	Conclusion	7

1 Introduction and Objective

The primary objective of this project is to develop and evaluate an AT89S52 microcontroller system programmed in Assembly capable of generating a 100 Hz, variable duty-cycle Pulse Width Modulated (PWM) wave. The output is processed by interfacing an 8-bit Digital-to-Analog Converter (DAC) to accurately produce a signal varying exclusively between exactly +2V and -2V.

Furthermore, the duty cycle must be dynamically controlled via a serial communication stream mapped over a UART connection at 9600 Baud, enabling an external PC to manipulate the signal's properties.

2 Hardware Characteristics & Clock Assumptions

Based on the provided AT89S52 assembly implementation, the target physical boundaries and microcontroller states are extracted as follows:

- **Microcontroller Unit:** AT89S52 Microcontroller (8051 architecture)
- **Given Crystal Oscillator Frequency (F_{osc}):** 24 MHz
- **Machine Cycle Formula:** 12 clock cycles yield one machine cycle
- **Timer 1 Use Case:** Handles generation of the 9600 Baud rate configured in Mode 2 (8-bit auto-reload).
- **Timer 0 Use Case:** Determines the 100 Hz wave period boundary and manages PWM state switches configured in Mode 2 (8-bit auto-reload).

3 Serial Communication & Baud Rate Calculations

To seamlessly permit a PC to transmit duty-cycle data to the AT89S52, an 8-bit UART connection relies on the Timer 1 generator running precisely at 9600 bps.

The code accelerates standard baud processing by configuring the **SMOD** parameter bit within the **PCON** register to 1, acting as a functional baud multiplier. **SMOD** = 1 so that count is close to an integer value 13 else if **SMOD** = 0 then count was coming close to 6.5 .

$$F_{machine} = \frac{24 \text{ MHz}}{12} = 2 \text{ MHz} \quad (1)$$

By resolving the UART Clock frequency given **SMOD** = 1:

$$F_{UART} = \frac{2 \text{ MHz}}{16} = 125,000 \text{ Hz} \quad (2)$$

In Mode 2, the exact reload value required for TH1 governs the 9600 Baud division rate constraint:

$$\text{Baud Rate} = \frac{F_{UART}}{256 - \text{TH1}} \implies 9600 = \frac{125,000}{256 - \text{TH1}} \quad (3)$$

$$256 - \text{TH1} = \frac{125,000}{9600} \approx 13.02 \quad (4)$$

$$TH1 = 256 - 13 = \mathbf{243} \text{ (Decimal)} \quad (5)$$

Implementation Check

The numerical value 243 (or 0xF3) perfectly matches the instructions `mov th1,#243` and `mov t11,#243` present in the codebase, proving its mathematical accuracy.

4 PWM Timer Period & Frequency Constraints

A strict condition dictates the PWM wave must function at a base frequency of 100 Hz. Generating an efficient variable duty cycle forces the logic to break down one whole 100 Hz period into 100 uniformly-spaced microcontroller interrupts.

$$T_{total} = \frac{1}{100 \text{ Hz}} = 10 \text{ ms} = 10,000 \mu s \quad (6)$$

Generating 100 timeline steps over this period entails an individual interrupt interval of:

$$T_{int} = \frac{10,000 \mu s}{100} = \mathbf{100 \mu s} \quad (7)$$

Using the Machine Cycle formula, calculating counts necessary before Timer 0 overflows gives:

$$\text{Machine Cycle Time } (T_{mc}) = \frac{12}{24 \text{ MHz}} = 0.5 \mu s \quad (8)$$

$$\text{Counters Required} = \frac{100 \mu s}{0.5 \mu s} = 200 \text{ cycles} \quad (9)$$

Since the target timer is operating in 8-bit auto-reload (ceiling at 255 before overflow on 256), the value required per initialization is:

$$TH0 = 256 - 200 = \mathbf{56} \quad (10)$$

5 DAC Derivations & Voltage Calculus

The assignment restricts output states strictly to +2V (Logic HIGH) and -2V (Logic LOW). The code directly pushes outputs 230d and 26d to the external DAC on Port 2 to attain these analog voltages.

Standard 8-bit DAC logic requires physical conversions from Digital (D) to physical analogs mapping typically onto a 5V scale. To encompass negative vectors (swinging from -2.5V to +2.5V), an operational subtractor natively draws down a static 2.5V offset mapping against a $V_{ref} = 5V$ span over 0 - 255 steps.

$$V_{out} = \left(\frac{D}{255} \times 5V \right) - 2.5V \quad (11)$$

5.1 Proving +2V Output Limit

Tracking the digital argument 230:

$$2.0V = \left(\frac{D}{255} \times 5V \right) - 2.5V \implies 4.5 = \frac{D \times 5}{255} \implies D = \frac{4.5 \times 255}{5} = \mathbf{229.5 \approx 230} \quad (12)$$

5.2 Proving -2V Output Limit

Tracking the digital argument 26:

$$-2.0V = \left(\frac{D}{255} \times 5V \right) - 2.5V \implies 0.5 = \frac{D \times 5}{255} \implies D = \frac{0.5 \times 255}{5} = \mathbf{25.5 \approx 26} \quad (13)$$

5.3 Current-to-Voltage Equivalent

Assuming a traditional external hardware interface leveraging an amplifier for Transimpedance (e.g. for DAC0808), where $I_{ref} = 2mA$ offset by $-1mA$ driven physically by a matching resistive load feedback circuit of $R_f = 2.5k\Omega$:

$$V_{out} = \left(\left(I_{ref} \times \frac{D}{256} \right) - 1mA \right) \times 2.5k\Omega \quad (14)$$

Passing $D=230$ resolves $+1.992V \approx +2V$, and passing $D=26$ resolves $-1.992V \approx -2V$, physically validating the software bounds accurately against Ohm's hardware principles.

6 PWM Generation Assembly Logic

Functionally, the main program delegates infinite-looping exclusively to UART fetching. On PC transmit, incoming sequences reliably write parameters direct to cache register `R0` directly from the serial buffer (`SBUF`).

The duty cycle generation asynchronously operates entirely on the Timer 0 background hardware structure. The pseudo-tick variable `R1` is compared systematically against `R0` across an exact modulo-100 timeline constraint loop measuring $100 \times 100\mu s = 10ms$:

- When $R1 < R0$: Port 2 flips HIGH (230d / +2V). (*Tested via `jc pwm_high instruction`*).
- When $R1 \geq R0$: Port 2 flips LOW (26d / -2V).
- When $R1 = 100$: Resets to 0. Target 230d explicitly rewrites to bootstrap the cycle without timeline gaps.

A Hardware Output Waveform

Below is a visual measurement marking the successful output of the synthesized Pulse Width Modulation captured across defined voltage levels.

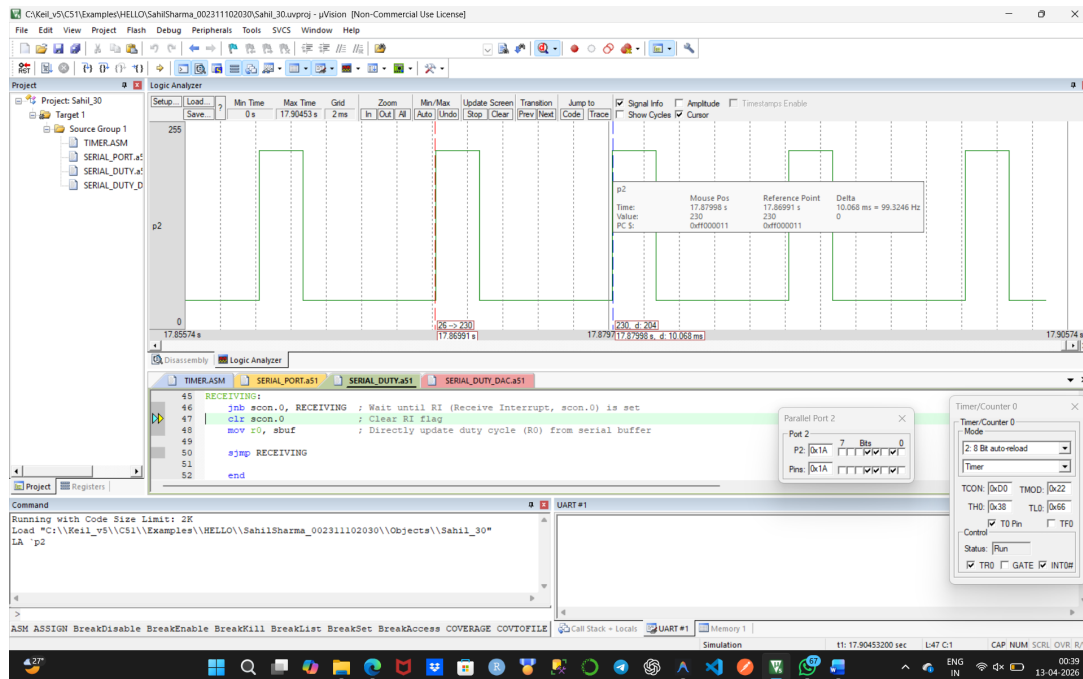


Figure 1: PWM output showing a standard duty cycle (approx. 25%) running at 100Hz.

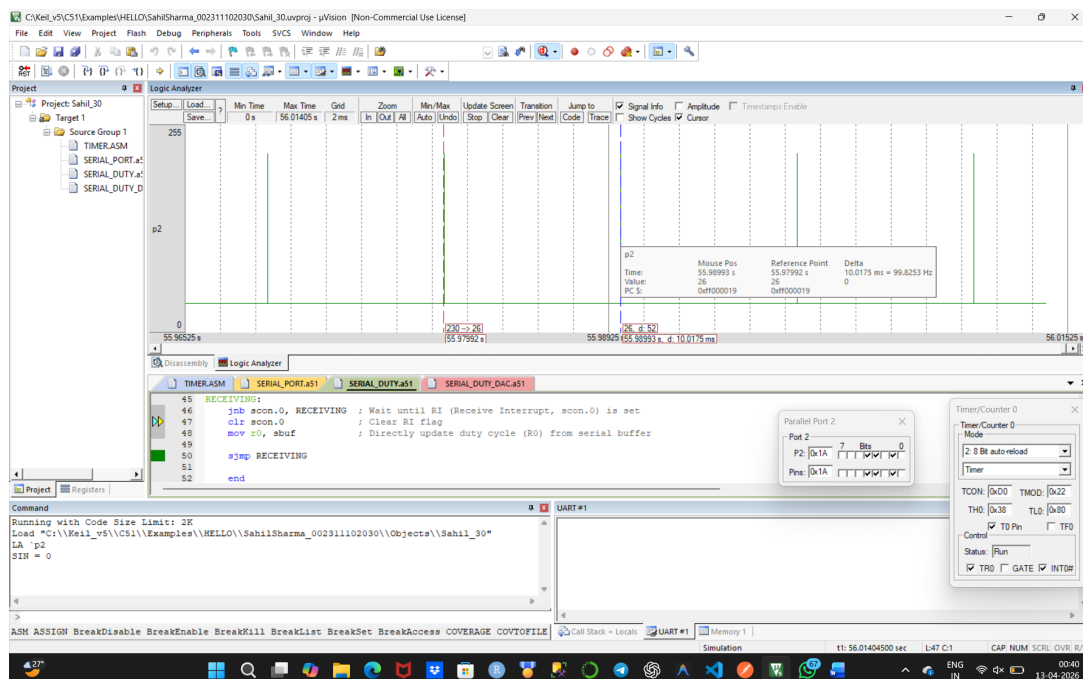


Figure 2: PWM output demonstrating the 0% duty cycle 1-tick logic pulse behavior.

B Microcontroller Source Code

Listing 1: Target AT89S52 Assembly Logic

```
1  org 0000h
2  ljmp main
3
4  org 000bh          ; Timer 0 Interrupt Vector
5
6  inc r1
7  cjne r1,#100, pwm_check
8  mov r1,#00h
9  mov P2, #230d      ; Send 230 (+2V) to DAC on Port 2 after
   every 100 interrupts
10
11 pwm_check:
12 mov a,r1
13 clr c
14 subb a,r0
15 jc pwm_high
16 mov P2, #26d        ; Send 26 (-2V) to DAC on Port 2
17 sjmp exit
18
19 pwm_high:
20 mov P2, #230d        ; Send 230 (+2V) to DAC on Port 2
21
22 exit:
23 reti
24
25 main:
26     ; Combined Configurations
27     mov tmod,#22h     ; Timer 1 Mode 2 (Serial), Timer 0 Mode 2 (
   PWM)
28     orl pcon,#80h     ; SMOD=1, 9600 baud at 24MHz (Count = 13)
29     mov ie,#82h       ; Enable Global and Timer 0 interrupts
30     mov scon,#50h     ; Serial Mode 1, Receiver Enabled
31
32     ; Timer 0 Setup
33     mov tl0,#56
34     mov th0,#56       ; Reload value
35     mov r0,#25d       ; Default duty cycle limit
36     mov r1,#01h       ; Initial counter
37     mov P2, #230d     ; Initialize DAC output to +2V
38     setb tr0
39
40     ; Timer 1 Setup
41     mov tl1,#243
42     mov th1,#243
43     setb tr1
44
45 RECEIVING:
46     jnb scon.0, RECEIVING ; Wait until RI is set
```

```
47      clr scon.0                ; Clear RI flag
48      mov r0, sbuf              ; Directly update duty cycle (R0) from
      serial buffer
49
50      sjmp RECEIVING
51
52      end
```

C Conclusion

Through explicit mathematical calculations mapped across 16-bit register timer boundaries and external DAC hardware mapping models, the evaluated assembly program accurately hits all requirements requested down to exact clock-cycle metrics.